Лабораторийн ажил №7

Түгжрэл (Deadlocks) — Бүлэг 8, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Түгжрэл үүсэх зайлшгүй дөрвөн нөхцөл (§8.3.1): **mutual exclusion** (нөөцийг нэг удаад нэг урсгал ашиглана), **hold and wait** (нөөц эзэмшсээр өөр нөөц хүлээнэ), **no preemption** (нөөцийг хүчээр булааж авахгүй), **circular wait** (тойрог хүлээлт). Дөрвүүлээ зэрэг биелбэл түгжрэл үүсэх боломжтой.

Дасгал 8.1

Асуулт. Компьютерийн системтэй холбоогүй түгжрэлийн гурван жишээ дурд.

Хариулт.

1. **Уулзвар дахь машинууд.** Гэрлэн дохиогүй дөрвөн замын уулзварт дөрвөн машин зэрэг орж ирээд тус бүр баруун талынхаа машиныг өнгөрөхийг хүлээвэл хэн нь ч хөдлөхгүй (тойрог хүлээлт). Зам (уулзварын хэсэг) нь дундаа хэрэглэх боломжгүй нөөц.
2. **Нэг эгнээтэй гүүр.** Эсрэг талаас хоёр машин гүүрийн дунд нүүр тулбал аль аль нь ухрахгүй бол хоёулаа зогсоно (Exercise 8.30-ын санаа).
3. **Хоёр хүн, хоёр хэрэгсэл.** Нэг хүн сав, нөгөө нь халбага барьж аль аль нь нөгөөгийнхөө барьсныг хүлээвэл хоол хийгдэхгүй. Мөн хоёр хүн утсаар нэгнээ залгахад хоёулаа ”завгүй” дуут дохио сонсож, нөгөө нь тасалж залгахыг хүлээх.

Дасгал 8.2

Асуулт. Систем аюулгүй биш төлөвт байна гэж үзье. Урсгалууд түгжрэлд орохгүйгээр гүйцэтгэлээ дуусгах боломжтойг харуул.

Хариулт.

Аюулгүй биш төлөв гэдэг нь хамгийн муу тохиолдолд (урсгал бүр Max-аа нэг дор шаардвал) түгжрэл үүсч болзошгүй гэсэн үг; түгжрэл заавал үүснэ гэсэн үг биш. Max нь зөвхөн дээд хязгаар бөгөөд урсгал түүнээс бага нөөц шаардаад дуусч болно, эсвэл нөөцөө эрт чөлөөлж болно.

Жишээ. 12 соронзон хальстай систем (§8.6.2): T_0 — Max 10, эзэмшиж буй 5; T_1 — Max 4, эзэмшиж буй 2; T_2 — Max 9, эзэмшиж буй 3; чөлөөтэй 2. Энэ төлөв аюулгүй биш: T_1 дуусаад 4 чөлөөтэй болсон ч T_0 (5 хэрэгтэй) болон T_2 (6 хэрэгтэй)-ын аль нь ч Max хүртлээ шаардвал хүрэлцэхгүй. Гэвч T_2 нэмэлт нөөц шаардалгүй ажлаа дуусгаж 3 хальсаа чөлөөлбөл (Max нь зөвхөн дээд хязгаар) чөлөөтэй 7 болж, T_0 5 хальс авч дуусна. Ингээд бүх урсгал түгжрэлгүй дуусна. Иймд аюулгүй биш төлөвөөс түгжрэлгүй дуусах гүйцэтгэлийн дараалал байж болно.

Дасгал 8.3

Асуулт. Системийн агшин зураг (A B C D нөөц):

	Allocation	Max	Available
T_0	0 0 1 2	0 0 1 2	1 5 2 0
T_1	1 0 0 0	1 7 5 0	
T_2	1 3 5 4	2 3 5 6	
T_3	0 6 3 2	0 6 5 2	
T_4	0 0 1 4	0 6 5 6	

Банкирын алгоритмаар: (a) Need матриц; (b) систем аюулгүй төлөвт байгаа эсэх; (c) T_1-ээс $(0, 4, 2, 0)$ хүсэлт ирвэл шууд хангаж болох уу?

Хариулт.

(a) Need = Max − Allocation:

	Need (A B C D)
T_0	0 0 0 0
T_1	0 7 5 0
T_2	1 0 0 2
T_3	0 0 2 0
T_4	0 6 4 2

(b) Аюулгүй байдлын алгоритм, Work = Available = $(1, 5, 2, 0)$:

Алхам	Нөхцөл $\text{Need}_i \le \text{Work}$	Шинэ Work = Work + Allocation_i
T_0	$(0,0,0,0) \le (1,5,2,0)$	$(1,5,3,2)$
T_2	$(1,0,0,2) \le (1,5,3,2)$	$(2,8,8,6)$
T_3	$(0,0,2,0) \le (2,8,8,6)$	$(2,14,11,8)$
T_4	$(0,6,4,2) \le (2,14,11,8)$	$(2,14,12,12)$
T_1	$(0,7,5,0) \le (2,14,12,12)$	$(3,14,12,12)$

Бүх урсгал дуусах $\langle T_0, T_2, T_3, T_4, T_1 \rangle$ дараалал байгаа тул систем **аюулгүйн төлөвт** байна.

(c) Хүсэлтийн алгоритм (§8.6.3.2): $\text{Request}_1 = (0,4,2,0) \leq \text{Need}_1 = (0,7,5,0)$ ✓ ба $\leq \text{Available} = (1,5,2,0)$ ✓. Хүсэлтийг өгсөн гэж үзвэл: $\text{Available} = (1,1,0,0)$, $\text{Allocation}_1 = (1,4,2,0)$, $\text{Need}_1 = (0,3,3,0)$. Аюулгүй байдлын алгоритм: $T_0 \to (1,1,1,2)$, $T_2 \to (2,4,6,6)$, $T_3 \to (2,10,9,8)$, $T_4 \to (2,10,10,12)$, $T_1 \to (3,14,12,12)$. Шинэ төлөв аюулгүй ($\langle T_0, T_2, T_3, T_4, T_1 \rangle$) тул хүсэлтийг **шууд хангаж болно**. (Тооцоог `lab7/banker.py`-ээр шалгасан.)

Дасгал 8.4

Асуулт. Түгжрэлээс сэргийлэх нэг арга: бусад бүх нөөцийн өмнө авах ёстой нэг "дээд эрэмбийн" нөөц F нэмэх (containment). A…E объектын аль нэгийн түгжээг авахын өмнө F-ийн түгжээг заавал авна. Энэ схемийг §8.5.4-ийн circular-wait схемтэй харьцуул.

Хариулт.

Containment (F түгжээ). Нэг урсгал F-ийг барьж байх үед өөр ямар ч урсгал A…E-ийн түгжээг авч чадахгүй тул A…E-ийн хувьд тойрог хүлээлт үүсэх боломжгүй — түгжрэлгүй. Гэвч F нь бүх зүйлийг дараалалд оруулдаг "том түгжээ" (coarse-grained lock) болно: A-тай ажиллах урсгал ба D-тэй ажиллах урсгал хамааралгүй байсан ч зэрэг ажиллаж чадахгүй. Зэрэгцээ ажиллагаа эрс буурч, F-ийн төлөөх өрсөлдөөн (contention) гацаа үүсгэнэ. Мөн урсгал хэр удаан F-ийг барихаас хамаарч хүлээлт урт болно.

Circular-wait схем (§8.5.4). Нөөц бүрт дугаар оноож, урсгалууд түгжээг зөвхөн өсөх дарааллаар авна. Түгжрэлгүй байдлыг мөн баталгаажуулна, мөн хамааралгүй нөөцүүдийг зэрэг ашиглах боломжтой тул **зэрэгцээ ажиллагаа илүү**. Сул тал нь: бүх кодыг дарааллын дүрмийг мөрдүүлэх хэцүү (жнь Figure 8.7-ийн `transaction()` шиг түгжээ динамикаар сонгогдох үед), том системд дарааллыг тогтоох, хадгалах ажил их, Linux-ийн `lockdep` шиг хэрэгслээр шалгах шаардлагатай.

Дүгнэлт: containment нь энгийн, алдаа гаргахад хэцүү боловч зэрэгцээ байдлыг алддаг; эрэмбэлсэн түгжээ нь гүйцэтгэл сайн боловч хэрэгжүүлэх, засварлахад сахилга бат шаарддаг.

Дасгал 8.5

Асуулт. §8.6.3-ийн аюулгүй байдлын алгоритм $m \times n^2$ эрэмбийн үйлдэл шаарддагийг батал.

Хариулт.

n — урсгалын тоо, m — нөөцийн төрлийн тоо.

1. Эхлүүлэх: Work = Available (m үйлдэл), $\text{Finish}[i]$ = false (n үйлдэл) — $O(m+n)$.
2. Нэг давталт: $\text{Finish}[i]$ =false ба $\text{Need}_i \leq$ Work байх i-г хайна. Урсгал

бүрийн хувьд хоёр вектор харьцуулах нь m үйлдэл, n урсгал шалгана → $O(m \cdot n)$. Олдсон урсгалын хувьд Work $+=$ Allocation$_i$ — $O(m)$.

3. Давталт бүрт дор хаяж нэг урсгал дуусна (эс бөгөөс алгоритм зогсоно), тиймээс давталт хамгийн ихдээ n удаа хийгдэнэ.

Нийт: $n \cdot (m \cdot n + m) = O(m \cdot n^2)$. Хамгийн муу тохиолдол: давталт бүрт олдох урсгал жагсаалтын хамгийн сүүлд байх үед бүх n урсгалыг дахин шалгах шаардлагатай тул энэ дээд хязгаарт хүрнэ.

Дасгал 8.6

Асуулт. Сард 5000 ажил гүйцэтгэдэг, түгжрэлээс сэргийлэх/гарах схемгүй систем. Сард 2 удаа түгжрэл үүсч, удаа бүр 10 ажлыг зогсоож дахин ажиллуулдаг. Ажил бүр $2 үнэтэй, зогсоосон ажлууд ойролцоогоор хагас гүйцэтгэгдсэн байдаг. Банкирын шиг түгжрэлээс гарах алгоритм суулгавал ажлын дундаж гүйцэтгэлийн хугацаа 10%-иар өснө; машин 30% сул тул бүх ажил гүйцэтгэгдэх ч эргэлтийн хугацаа дунджаар 20%-иар өснө. (a) Суулгахын талд, (b) эсрэг талд ямар үндэслэл байна?

Хариулт.

Алдагдлын тооцоо: сард $2 \times 10 = 20$ ажил дахин ажиллана, хагас гүйцэтгэгдсэн тул алдагдал $20 \times \$2 \times 0.5 = \$20$/сар. Харин алгоритмын нэмэлт ачаалал: $5000 \times \$2 \times 10\% = \$1000$/сарын CPU цаг.

(a) Суулгахын талд:

- Түгжрэл огт үүсэхгүй — оператор гараар илрүүлж, ажил зогсоож, дахин ажиллуулах шаардлагагүй (хүний цаг, алдаа гарах эрсдэл буурна).
- Урьдчилан таамаглах боломжтой байдал: зогсоосон ажлын хагас дутуу үр дүн (файл, өгөгдлийн сангийн өөрчлөлт)-ийг буцаах асуудал үүсэхгүй.
- Машин 30% сул тул нэмэлт 10% ачааллыг даана, бүх ажил гүйцэтгэгдсээр байна.

(b) Эсрэг талд:

- Түгжрэлийн алдагдал ердөө $\approx \$20$/сар, харин алгоритмын зардал $\approx \$1000$/сар — эдийн засгийн хувьд ашиггүй.
- Эргэлтийн хугацаа бүх 5000 ажилд 20% өснө — сард 20 ажлын саатлаас хамаагүй их хэрэглэгчид нөлөөлнө.
- Банкирын алгоритм урсгал бүр Мах-аа урьдчилан мэдэгдэхийг шаарддаг; бодит ажлуудад үүнийг тодорхойлоход хэцүү, нөөцийг хэтрүүлэн нөөцөлж ашиглалтыг бууруулна.
- Сул 30% нь ирээдүйн өсөлтөд зориулсан нөөц байж болох тул түүнийг алгоритмд зарцуулах нь эрсдэлтэй.

Энэ тохиолдолд түгжрэл ховор, алдагдал бага тул ихэнх ерөнхий зориулалтын OS (Linux, Windows)-ийн адил түгжрэлийг үл тоох / илрүүлж сэргээх хандлага илүү тохиромжтой.

Дасгал 8.7

Асуулт. Систем зарим урсгалаа хомсдолд (starvation) орсныг илрүүлж чадах уу? ”Тийм” бол хэрхэн, ”үгүй” бол хомсдлыг хэрхэн шийдвэрлэх вэ?

Хариулт.

Ерөнхийдөө үгүй — найдвартай илрүүлэх боломжгүй. Хомсдол гэдэг нь урсгал хязгааргүй удаан хүлээх явдал; тодорхой агшинд урсгал удаан хүлээж байгааг харж болох ч удахгүй нөөц авах эсэхийг мэдэх нь ирээдүйн хуваарилалтыг таамаглахыг шаардана. Түгжрэлээс ялгаатай нь хомсдолд мөчлөг (cycle) гэх мэт бүтцийн шинж тэмдэг байхгүй.

Практикт: хүлээлтийн хугацааг хянаж, тогтоосон хязгаараас хэтэрвэл ”хомсдолд орсон байж магадгүй” гэж үзэж болно (heuristic). Хомсдлыг шийдвэрлэх арга:

- **Хөгшрөлт (aging):** удаан хүлээсэн урсгалын эрэмбийг аажмаар нэмэгдүүлнэ (5-р бүлэг).
- **FIFO / ээлжийн дараалал:** нөөцийг хүсэлт ирсэн дарааллаар өгөх — хязгаарлагдмал хүлээлт (bounded waiting).
- **Түгжрэлээс сэргэхэд:** victim сонгохдоо урсгалыг хэдэн удаа буцаасныг (rollback) зардлын хүчин зүйлд оруулж, нэг урсгалыг дахин дахин сонгохгүй байх (§8.8.2).
- Exercise 8.31 / `bridge.c fair` горим: нэг талаас дараалан хамгийн ихдээ K хүн оруулаад ээлжийг солих.

Дасгал 8.8

Асуулт. Бодлого: нөөцийн хүсэлтийг шууд хангаж чадахгүй бол хүлээж (blocked) буй урсгалуудаас хүссэн нөөцийг нь булааж хүсэгч урсгалд өгнө; хүлээж буй урсгалын хүлээх вектор булаагдсан нөөцөөр нэмэгдэнэ. (a) Түгжрэл үүсч болох уу? (b) Хязгааргүй хаагдал (indefinite blocking) үүсч болох уу?

Хариулт.

(a) Түгжрэл үүсэхгүй. Энэ бодлого **no preemption** нөхцөлийг зөрчдөг: хаагдсан (blocked) урсгалын эзэмшиж буй нөөцийг булааж авдаг. Хаагдсан урсгал нөөцөө ”барьсаар” хүлээж чадахгүй тул тойрог хүлээлт тогтвортой хэвээр үлдэхгүй — хэрэв T_i нь T_j-ийн нөөцийг хүлээж, T_j хаагдсан бол T_i тэр нөөцийг шууд булааж авна. Ажиллаж буй (хаагдаагүй)

урсгалын нөөцийг булаахгүй ч ажиллаж буй урсгал хүлээхгүй тул мөчлөгт оролцохгүй.

(b) Тийм, хязгааргүй хаагдал (хомсдол) үүсч болно. Хаагдсан урсгалаас нөөц байнга булаагдаж болно: жишээнд T_0 $(0,0,1)$-ийг хүлээж хаагдсан, тэр хооронд T_2 түүний нөөцийг авсан. Хэрэв бусад урсгалууд үргэлжлэн шинэ хүсэлт гаргаж T_0-ийн нөөцийг дахин дахин булааж байвал T_0-ийн хэрэгцээ хэзээ ч бүрэн хангагдахгүй — хүлээх вектор нь улам томорсоор байна.

Дасгал 8.9

Асуулт. Системийн агшин зураг:

	Allocation (A B C D)	Max (A B C D)
T_0	3 0 1 4	5 1 1 7
T_1	2 2 1 0	3 2 1 1
T_2	3 1 2 1	3 3 2 1
T_3	0 5 1 0	4 6 1 2
T_4	4 2 1 2	6 3 2 5

Банкирын алгоритмаар дараах төлөв бүр аюулгүй эсэхийг тогтоо: (a) Available $= (0,3,0,1)$; (b) Available $= (1,0,0,2)$.

Хариулт.

Need = Max − Allocation: $T_0 = (2,1,0,3)$, $T_1 = (1,0,0,1)$, $T_2 = (0,2,0,0)$, $T_3 = (4,1,0,2)$, $T_4 = (2,1,1,3)$.

(a) Available $= (0,3,0,1)$ — аюулгүй БИШ.

Урсгал	Need $\le$ Work?	Work
T_2	$(0,2,0,0) \le (0,3,0,1)$ ✓	$(3,4,2,2)$
T_1	$(1,0,0,1) \le (3,4,2,2)$ ✓	$(5,6,3,2)$
T_3	$(4,1,0,2) \le (5,6,3,2)$ ✓	$(5,11,4,2)$
T_0	$(2,1,0,3)$: D $3 > 2$ ×	—
T_4	$(2,1,1,3)$: D $3 > 2$ ×	—

T_2, T_1, T_3 дууссаны дараа D нөөц ердөө 2 үлдэх ба T_0, T_4 тус бүр 3 D шаарддаг тул аль нь ч дуусч чадахгүй. Иймд бүх урсгал дуусах дараалал байхгүй — төлөв аюулгүй биш.

(b) Available $= (1,0,0,2)$ — аюулгүй. T_1: $(1,0,0,1) \le (1,0,0,2)$ → Work $= (3,2,1,2)$; T_2: $(0,2,0,0) \le$ → $(6,3,3,3)$; T_3: $(4,1,0,2) \le$ → $(6,8,4,3)$; T_4: $(2,1,1,3) \le$ → $(10,10,5,5)$; T_0: $(2,1,0,3) \le$ → $(13,10,6,9)$. Аюулгүй дараалал: $\langle T_1, T_2, T_3, T_4, T_0 \rangle$. (`banker.py`-ээр шалгасан.)

Дасгал 8.10

Асуулт. Аюулгүй байдлын алгоритмын кодоо ашиглаж, $\text{Max}_i = \text{Waiting}_i + \text{Allocation}_i$ гэж тодорхойлж түгжрэл илрүүлэх алгоритм хэрэгжүүлж болох уу?

Хариулт.

Тийм. Түгжрэл илрүүлэх алгоритм (§8.7.2) нь аюулгүй байдлын алгоритмтай бараг ижил бүтэцтэй бөгөөд Need-ийн оронд одоогийн Request (хүлээж буй) матрицыг ашигладаг. $\text{Max}_i = \text{Waiting}_i + \text{Allocation}_i$ гэж тодорхойлбол $\text{Need}_i = \text{Max}_i - \text{Allocation}_i = \text{Waiting}_i$ болох тул аюулгүй байдлын алгоритм яг "одоо хүлээж буй хүсэлтээ авбал урсгал дуусч чадах уу?" гэдгийг шалгана — энэ нь илрүүлэх алгоритмын агуулга.

Ялгааг анхаарах: илрүүлэх алгоритмд $\text{Allocation}_i = 0$ урсгалыг эхнээсээ Finish=true гэж үздэг (энэ нь $\text{Waiting}_i \le \text{Work}$ нөхцөлтэй ихэвчлэн ижил үр дүн өгнө). Мөн тайлбар нь өөр: аюулгүй байдлын алгоритм "үгүй" гэвэл аюулгүй биш төлөв (түгжрэл үүсч болзошгүй), харин энд "үгүй" бол Finish=false урсгалууд одоо түгжрэлд орсон гэсэн үг — учир нь урсгалууд хүлээж буй хүсэлтээсээ илүүг шаардахгүй гэсэн өөдрөг таамаглалтай.

Дасгал 8.11

Асуулт. Зөвхөн нэг урсгалтай (single-threaded) нэг процессоос бүрдсэн түгжрэл байж болох уу? Тайлбарла.

Хариулт.

Тодорхойлолтын дагуу үгүй. Түгжрэл үүсэхэд circular wait буюу хоёр ба түүнээс дээш урсгал бие биенийхээ эзэмшсэн нөөцийг хүлээх хэрэгтэй. Нэг урсгалтай процесс нөөцийг эзэмшиж байх үедээ хүлээж байвал, тэр нөөцийг чөлөөлөх өөр урсгал байхгүй тул энэ нь мөчлөг биш.

Гэхдээ практикт өөртөө гацах (self-deadlock) боломжтой: жишээ нь рекурсив биш мутексийг аль хэдийн эзэмшсэн урсгал дахин `pthread_mutex_lock()` дуудвал үүрд хүлээнэ; эсвэл семафорыг 0 үед `wait()` хийгээд `signal()`-ийг өөрөө дараа нь хийхээр бичвэл. Энэ нь программын алдаа бөгөөд нөөц хуваарилалтын графын утгаар хоёр урсгалын түгжрэл биш.

Programming Problems (бичлэгт)

8.32 — нэг эгнээтэй гүүр (`lab7/bridge.c`). POSIX урсгал: хойд ба өмнөд зүгийн тариаланч тус бүр урсгал. `pthread_mutex` ба `pthread_cond`: гүүрэн дээр зөвхөн нэг чиглэлийнхэн байна; эсрэг чиглэлийнх гүүр хоосрохыг хүлээнэ — эсрэг талын хоёр тариаланч зэрэг гүүрэнд орох (түгжрэл) боломжгүй. Гүүрээр явах хугацаа санамсаргүй (`usleep`). `fair` горим (Exercise 8.31): нэг

талаас дараалан хамгийн ихдээ 2 хүн орсны дараа эсрэг талд ээлж өгч хомсдлоос сэргийлнэ. Программ төгсгөлд ”эсрэг чиглэл зэрэг гүүрэн дээр: 0 удаа” гэж шалгана; 30 удаагийн давталтад 0.

8.33 — түгжээг дараалаар авах (`lab7/Transaction.java`). Figure 8.7-ийн `transaction(from, to)`: хоёр урсгал $A \to B$ ба $B \to A$ зэрэг хийвэл тойрог хүлээлт үүсч гацна (`naive` горим 3 с-ийн дараа ”DEADLOCK” гэж илрүүлдэг). Засвар: `System.identityHashCode()`-ийн бага утгатай дансны түгжээг эхэлж авна; хэш тэнцүү бол нэмэлт `tieLock` авна. Үр дүн: 400 гүйлгээ бүгд дуусч, нийт үлдэгдэл 2000 хэвээр.